

CS 423

Operating System Design:
COW FS & Remote Access

Apr 28

Ram Kesavan

LOGISTICS

Midterm papers: pickup from my office during OH
MP4 due by **May 6, 11:59 CT**

Last lecture: 5/5 will be a review

Finals: 5/13 at 7-10pm, DCL 1320 (next door)

AGENDA / LEARNING OUTCOMES

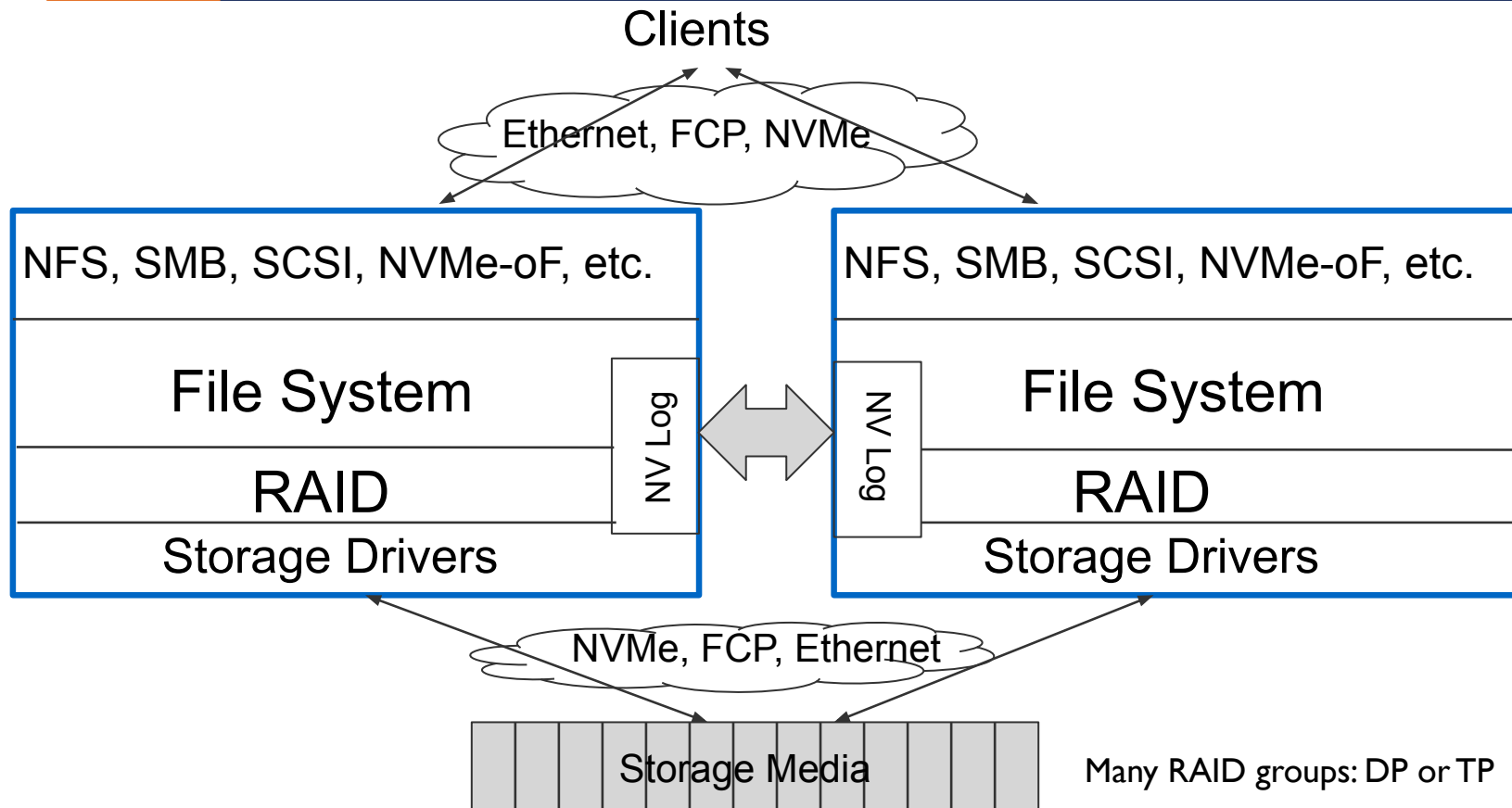
Enterprise-grade COW File Systems

Eg. WAFL, ZFS, BtrFS

Remote Access of File Systems

RECAP

ENTERPRISE-GRADE 2-NODE HA SYSTEM





WAFL file system is a tree of blocks

everything is a file, including all metadata

metadata files have well-known inums

each dirty buffer written to new (free) block in storage, except superblock

Modify op:

dirty in-memory state

creates log entry in NVRAM, is mirrored to partner's NVRAM

then ack'ed

COW file system: so everything is written to a new spot in persistent storage (except superblock)

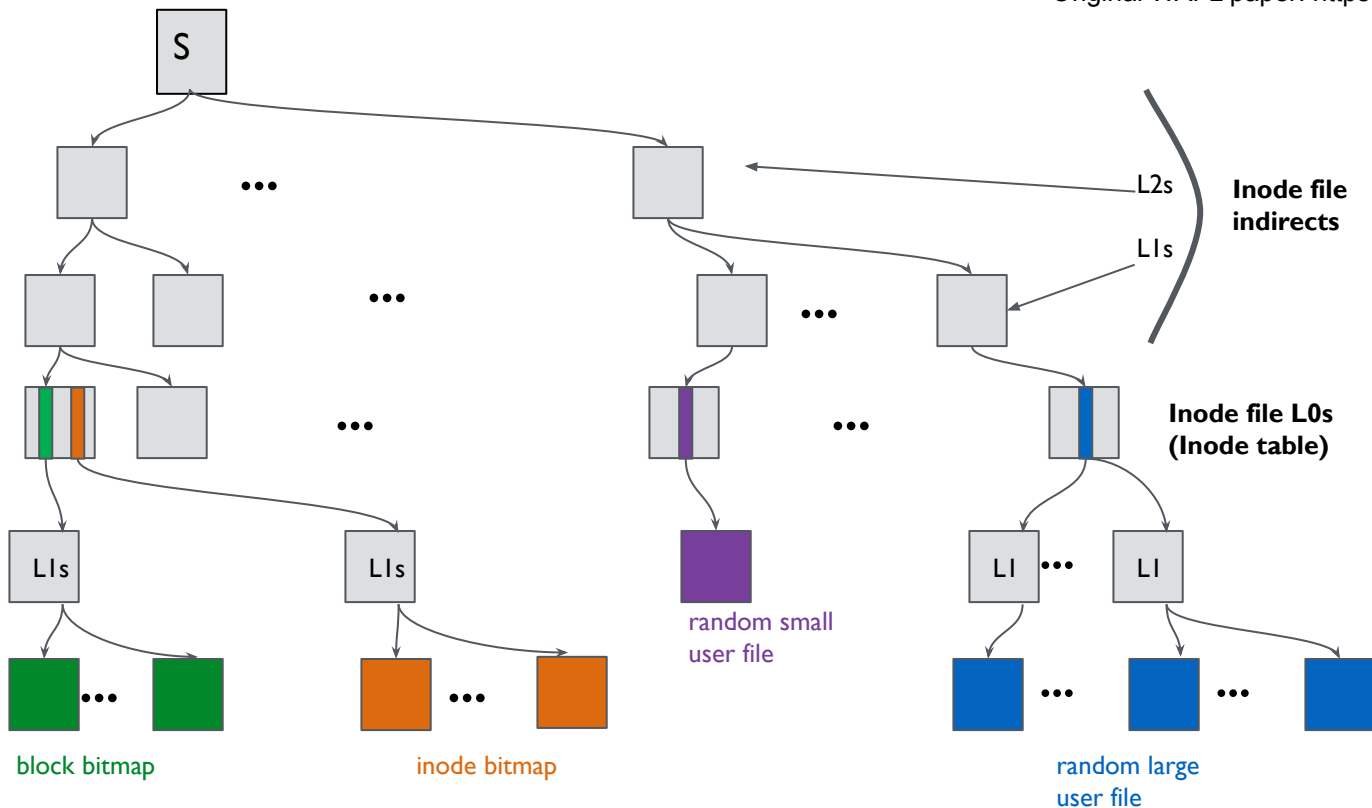
Large checkpoints: ~GB worth of dirty buffers

~2s to 5s long

all dirty buffers can be persisted in any order; superblock written last

COW: so persistent file system is always self-consistent

Original WAFL paper: <https://rcs.uwaterloo.ca/papers/wafl.pdf>



Everything is a file

All indirects have the same fanout (#ptrs)

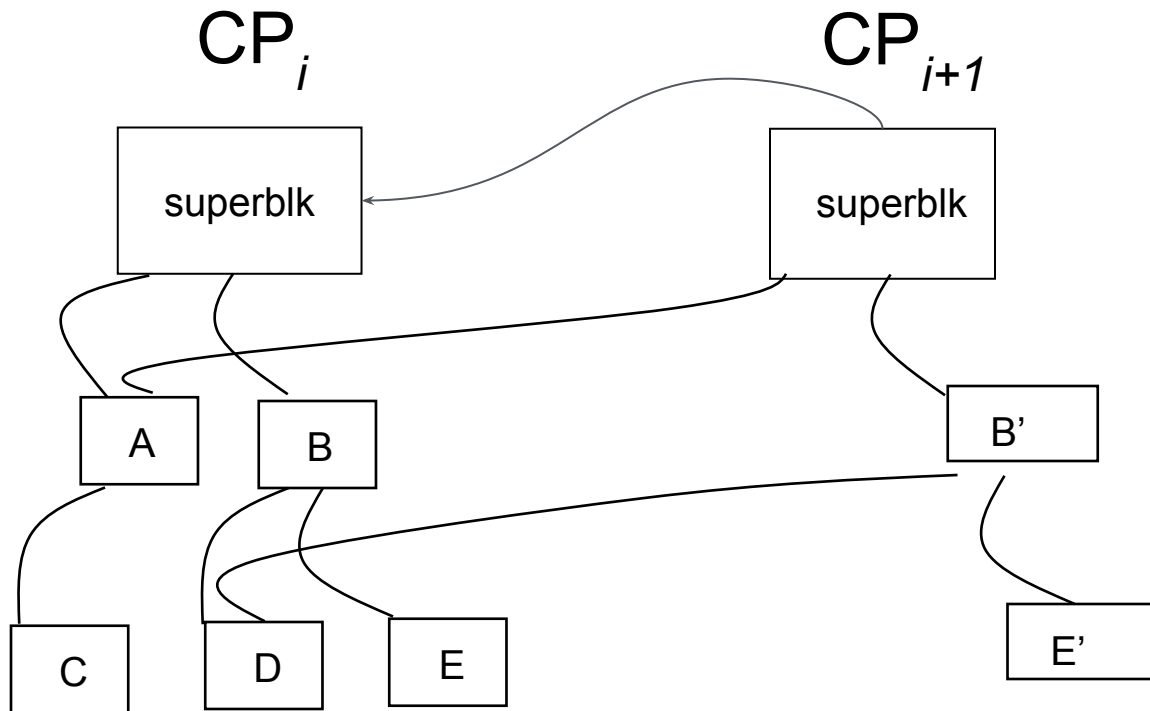
File trees are symmetric
& balanced

4 GiB of storage
= 4GiB/4KiB blocks
= 1M/32k bits-per-blk
= 32 bitmap blocks

4 TiB of storage
= 32k bitmap blocks

END RECAP

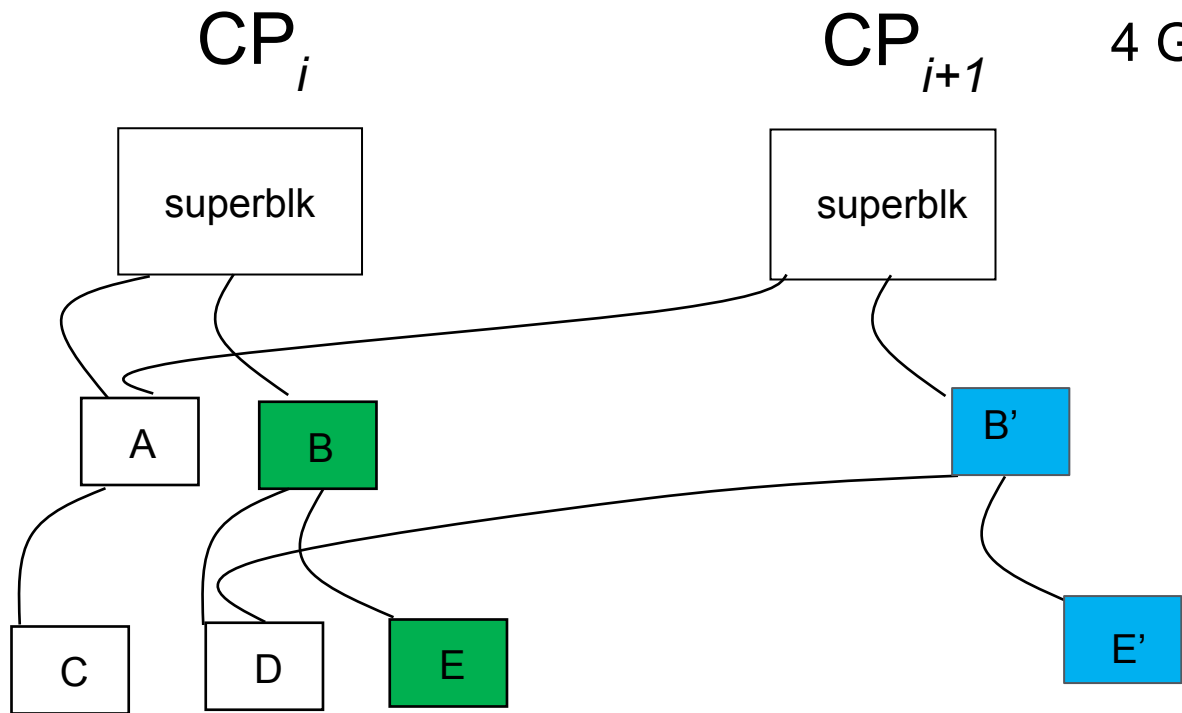
COW: Atomic Checkpoint



CHECKPOINTS, JOURNAL, & DOUBLE BUFFERING



Block Bitmap Updates



4 GB/s writes need

- 1 M/s allocations of **new blocks**
- 1 M/s frees of **unused blocks**

[ACM paper](#)

Block Bitmap: used(1)→free(0)



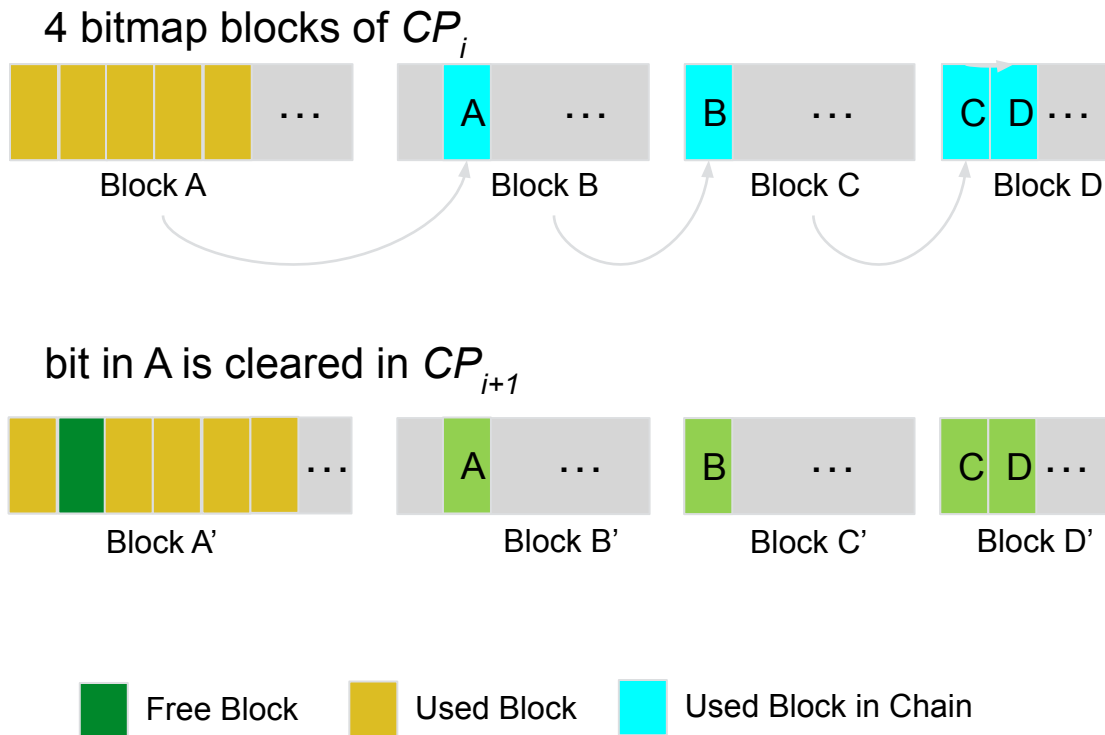
Long CPs problem:

- with larger bitmaps
- random frees → more dirty bitmap blocks
- worsened by long bitmap “chains”

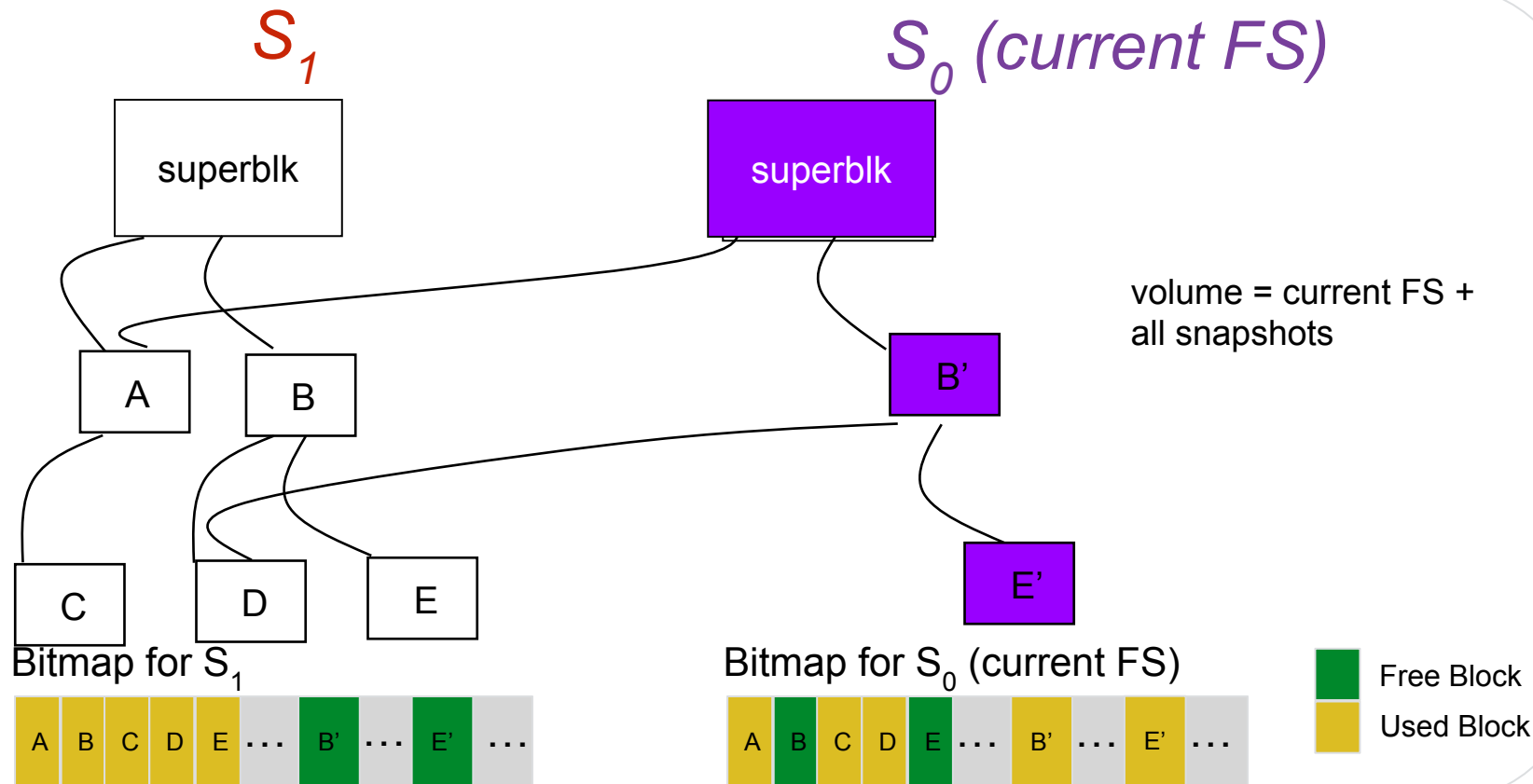
Long CPs hurt file system write throughput

- too many dirty bufs in memory
- journal gets filled up

Solved: [ACM paper](#)



SNAPSHOTS





Modern COW file systems (WAFL, ZFS, BtrFS)

- Are log-structured like LFS

- But keep a separate journal to guarantee zero data loss

Enterprise-grade storage systems

- Can be built using COW file systems

- Large checkpoints→efficient sequential writes to RAID4-style storage

Enterprise-grade features:

- High performance, reliability, mirroring, snapshots, clones, compression, deduplication, worm, encryption, tiering, replication, etc.



Remote File Access

Client-Server architecture
NFS/SMB

Distributed File Systems

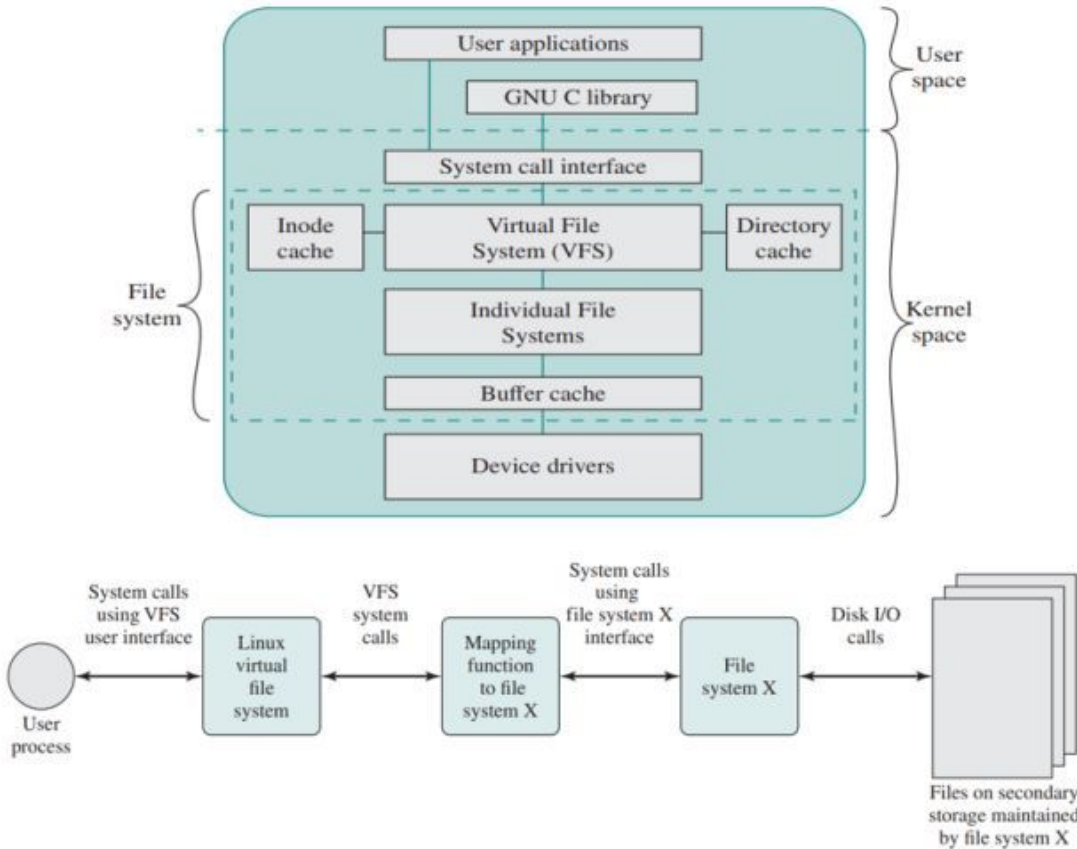
GFS + Colossus

Conflation of these 2 terms:

“remote file access”: many clients → a single server exports a file system

“distributed file systems”: multiple servers collaborate to export a single file system

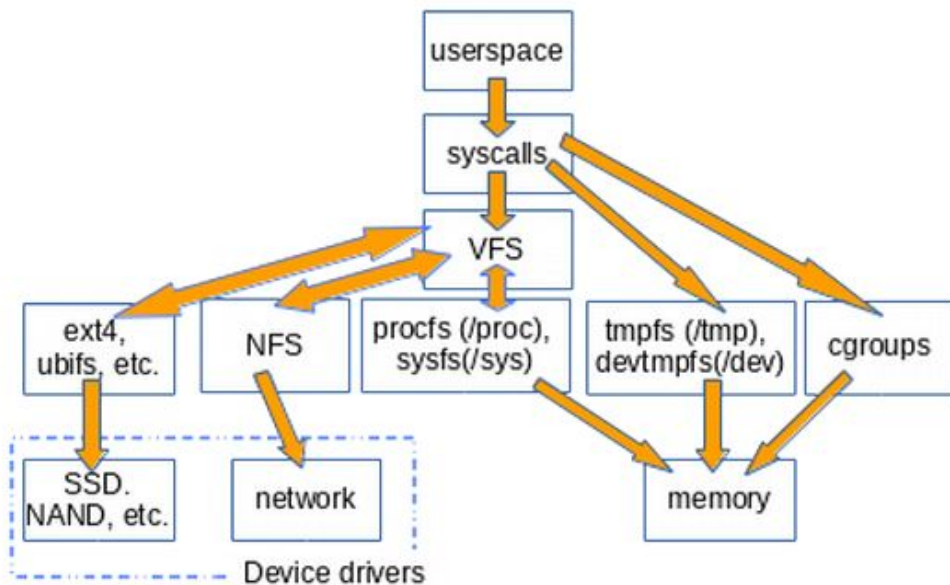
VIRTUAL FILE SYSTEM LAYER



VFS

- used by many OSes
- abstraction layer to different FS types
- VFS translates each system call into the underlying FS's API
- maintains info from all FSs in the caches—inode, dirent, buffer.
- Allows a single machine to mount many file systems (of different types)

VIRTUAL FILE SYSTEM LAYER



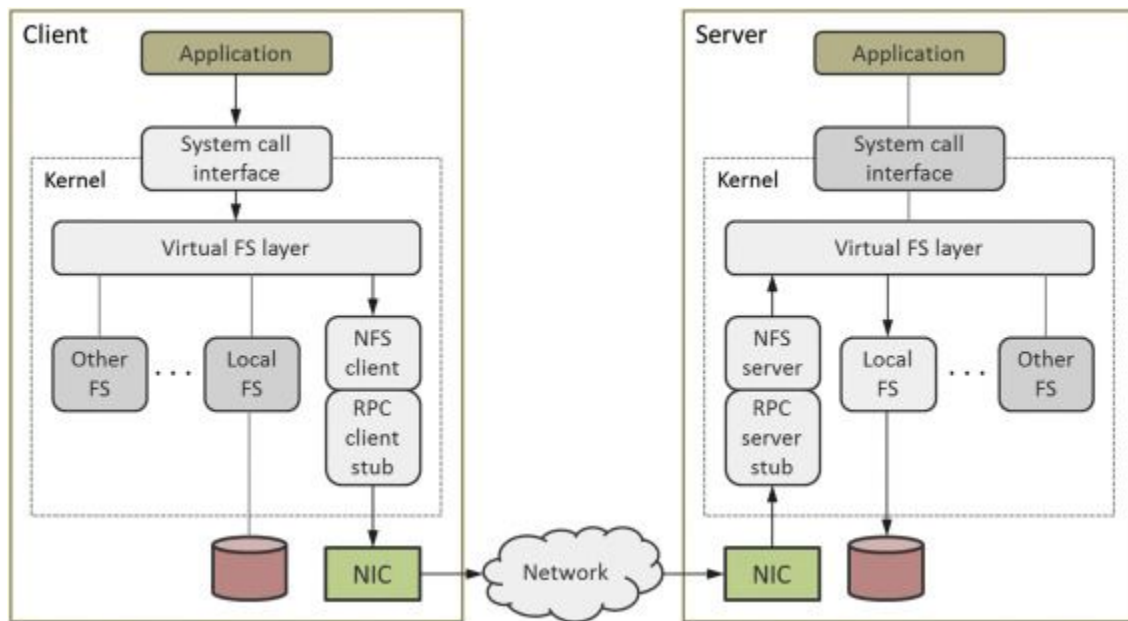
Local

- File systems on locally attached HDDs & SSDs
- File systems on local DRAM: do not survive crashes

Remote

- Protocol-specific client that talks with remote file service
- Protocols:
 - NFS: Sun
 - SMB: Microsoft
 - AFS: CMU (legacy)

NETWORK FILE SYSTEM (NFS)



- Client-server model
- Client allocates a socket and establishes communication over TCP or UDP
- Uses remote procedure call (RPC) abstraction to send & receive messages
- NFS protocol messages

NFS V2 GOAL: SIMPLE CRASH RECOVERY



- Many clients to a single server
- Client or server could crash
- A protocol can be *stateless* or *stateful*
- Stateless: server has no info about clients' states
 - Crash recovery becomes easy*
 - Server simply reboots/restarts
 - Each NFSv2 message must include all requisite info
 - Discuss: `open()` system call semantics
 - `fd` returned must work until `close()`
 - offset must be tracked

**ignore atomicity of file system updates: journaling handles that*

OPEN SYSCALL: STATELESSNESS



- Client tracks much of it
 - offset, file descriptor mapping, etc.
- `open()`: does client need to send NFS message to server?
 - if no, why?
 - if yes, what should server respond with?

OPEN SYSCALL: SERVER RESPONSE



- A file identifier that survives server restarts
 - `<volume ID, inode#>`
 - volume ID: unique file system ID
 - typically persisted in superblock

Client A

```
1. fdA = open("foo/bar", O_RDONLY);  
// sends NFS messages
```

```
4. read(fdA, buf, 4096);  
// sends NFS read for <v,501> at offset 0
```

```
6. write(fdA, buf, 1024);  
// sends NFS write for <v,501> at offset 4K
```

Server

```
2. // inum of "bar" = 501  
respond with <v, 501>  
3. server crashes & restarts
```

```
5. ack // returns 4K of data
```

```
7. ack // writes 1K at offset 4K
```

OPEN SYSCALL: SERVER RESPONSE



- A file identifier that survives server restarts
- But...

Client A

1. fdA = open("foo/bar", O_RDONLY);

9. read(fdA, buf, 4096);

Client B

3. unlink("foo/bar");

5. fdB = open("foo/baz", O_CREAT);

7. write(fdB, buf, 4096);

Server

2. // inum of "bar" = 501
respond with <v, 501>

4. // inum 501 is now free

6. // 501 is reused for baz
respond with <v, 501>

8. ack

10. returns baz's data!

OPEN SYSCALL: WITH GENERATION#



- A file identifier that survives server restarts
- But...

Client A

1. fdA = open("foo/bar", O_RDONLY);

9. read(fdA, buf, 4096);
// NFS read with <v, 501, g>

Client B

3. unlink("foo/bar");

5. fdB = open("foo/baz", O_CREAT);

7. write(fdB, buf, 4096);

Server

2. // inum of "bar" = 501
respond with <v, 501, g>

4. // inum 501 is now free

6. // 501 is reused for baz
respond with <v, 501, g+1>

8. ack

10. responds with ESTALE

NFS V2 CLIENT → SERVER MESSAGE



open(): LOOKUP(s) to walk path + GETATTR

 O_TRUNC: SETATTR (to truncate file to zero length)

 O_CREAT: CREATE (to create a new file)

read(): READ

write(): WRITE

unlink(): REMOVE

readdir(): READDIR

 returns (subset of) dir entries (+ opaque resumption cookie)

mkdir(): MKDIR

rmdir(): RMDIR

close(): nothing!*

*2 slides later



Several failure possibilities

- NFS request lost

- Server is down/restarting

- NFS response is lost

Idempotency simplifies failure handling

- Client retries request after timeout

- Non-modify ops are trivially idempotent

- WRITE is also idempotent

- But, life is not perfect

 - REMOVE, MKDIR, RMDIR



Client caches data & metadata

Obvious benefit for non-modify ops

Also caches results of write (aka **write buffering**)

`fsync()` forces a flush to server

Problems

1. ClientA's read can't see results from ClientB's write

flush-on-close semantics: `close()` flushes buffered writes

subsequent reads see updated content

2. Stale content in ClientA's cache

client sends `GETATTR` to server

response includes file's last-modify-timestamp

check if its equal to attributes cached in client

if not, invalidate cache contents, fetch from server



Server caches data & metadata

Can respond without accessing storage devices

Problem

Delay in persisting modify ops

Solved by journaling

NFS has been a huge success

Is the dominant file access protocol

Many companies: Sun, NetApp, EMC, IBM sell/sold NFS servers



AFS

Built by CMU; obsolete

Client `open()` fetches + caches entire file

NFS caches only fetched blocks of file

SMB

Stateful; connection-oriented

Server tracks clients, open files & locks

More complex

Subsequent NFS versions

v2: now obsolete

v3: better performance, larger file sizes (>2GiB)

v4: stateful; connection-oriented, improved security, locking

v4.1: pNFS - client can read/write directly from multiple servers
works with distributed NFS service



Distributed File Systems

GFS + Colossus